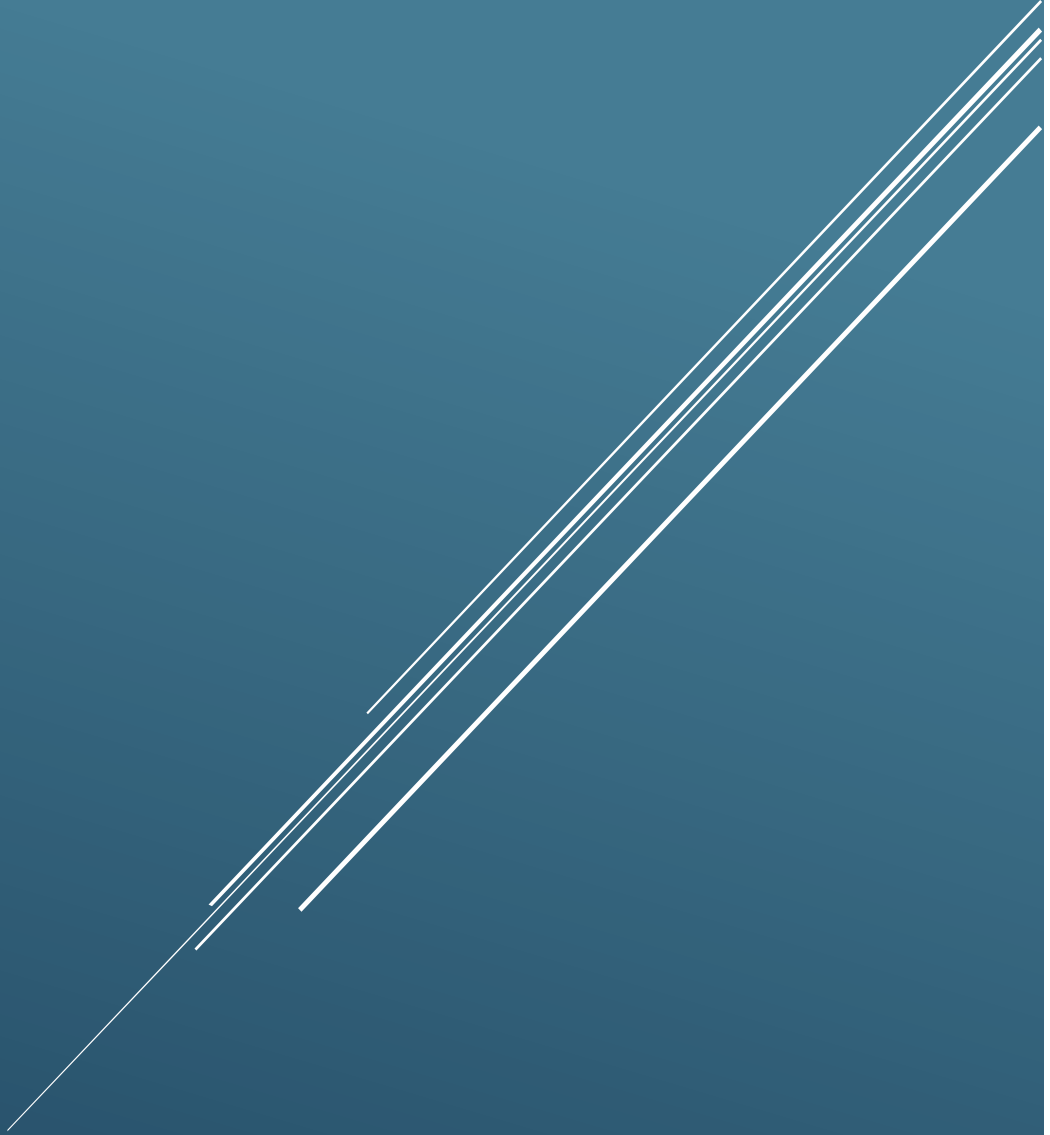


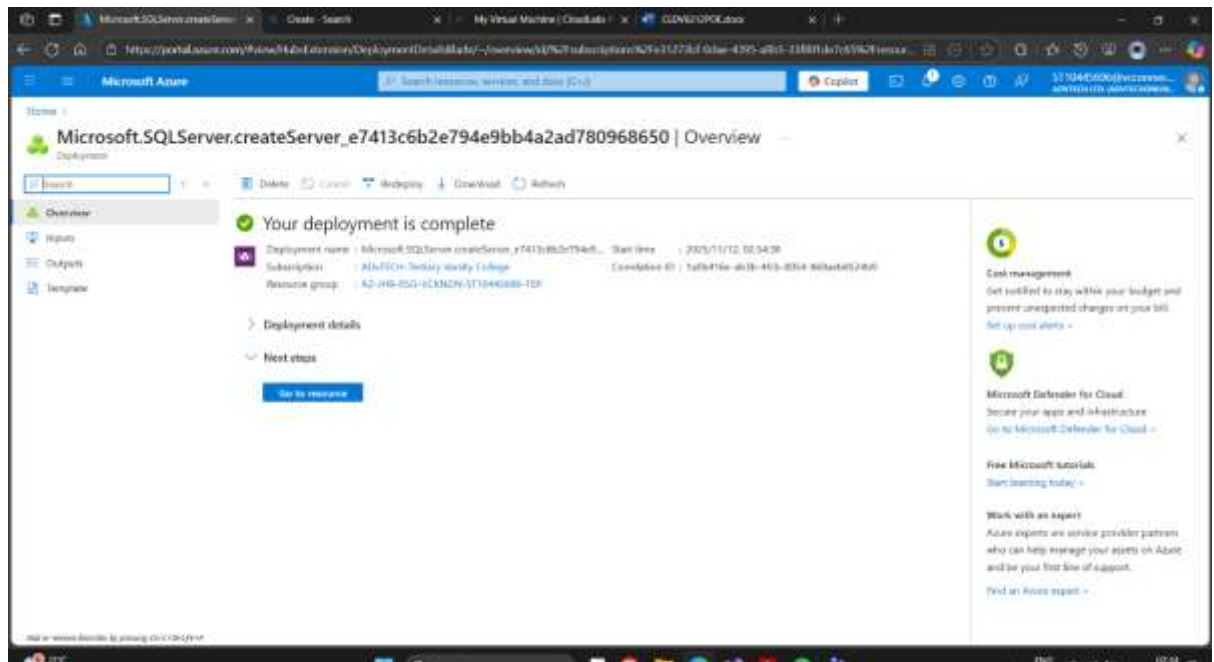


CLDV_POE_P3

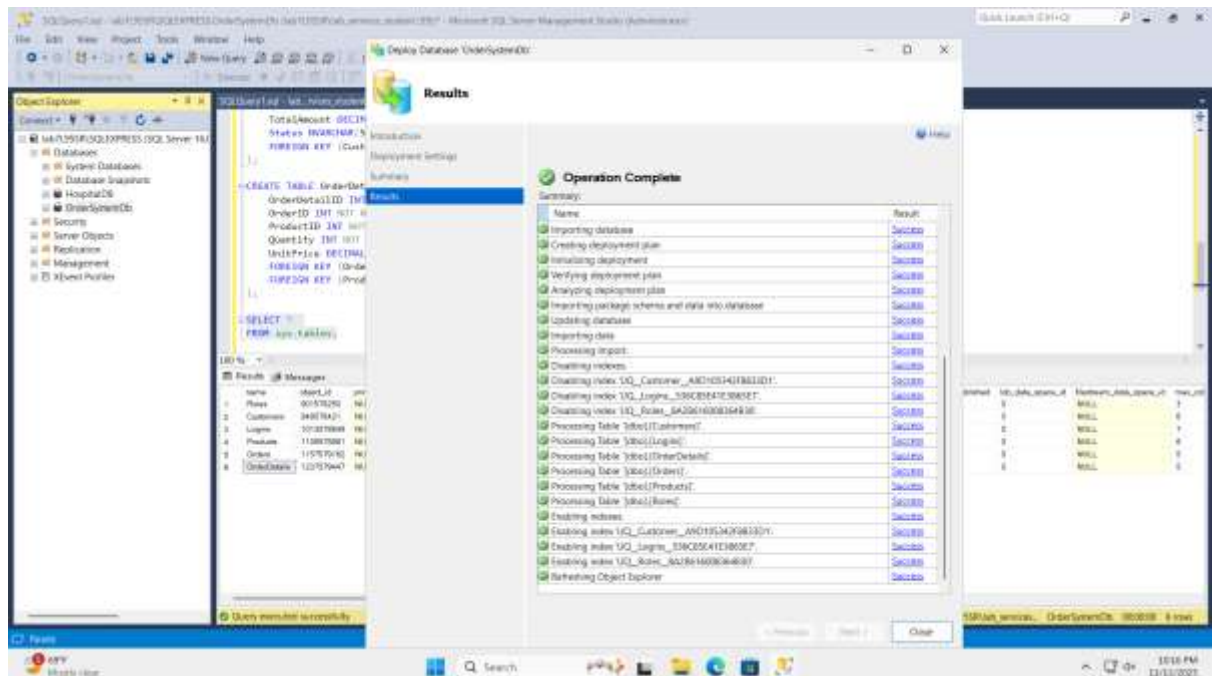
ST10445696

A) Creating an azure database

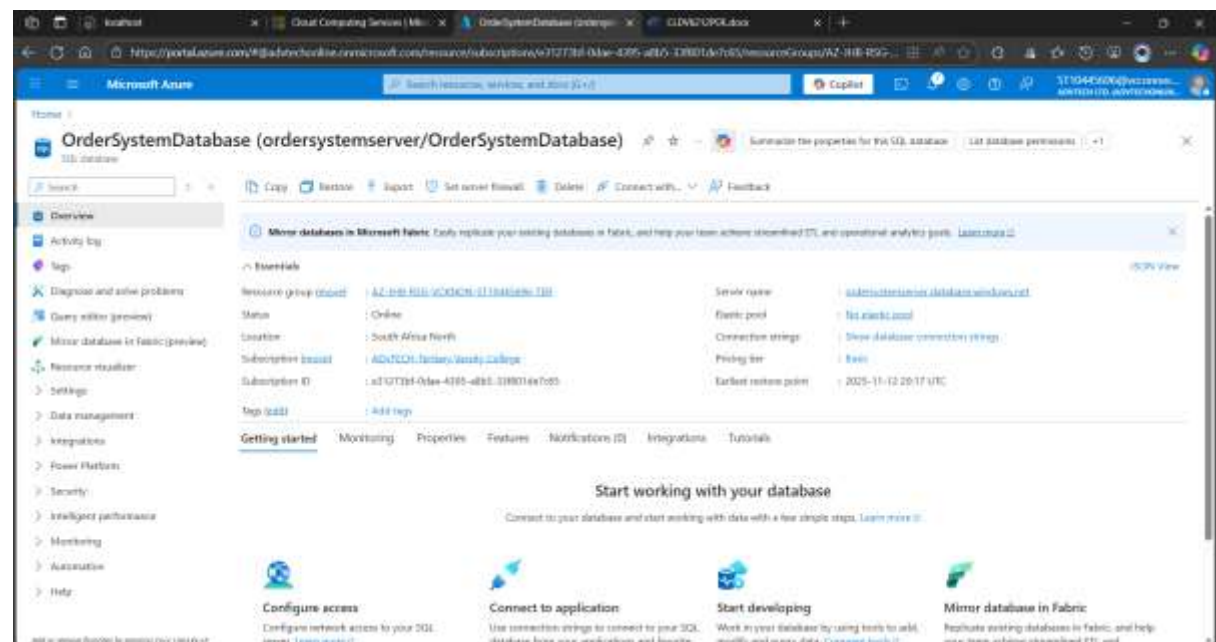
1. Successful Server Deployment



2. Successful db deployment to Azure



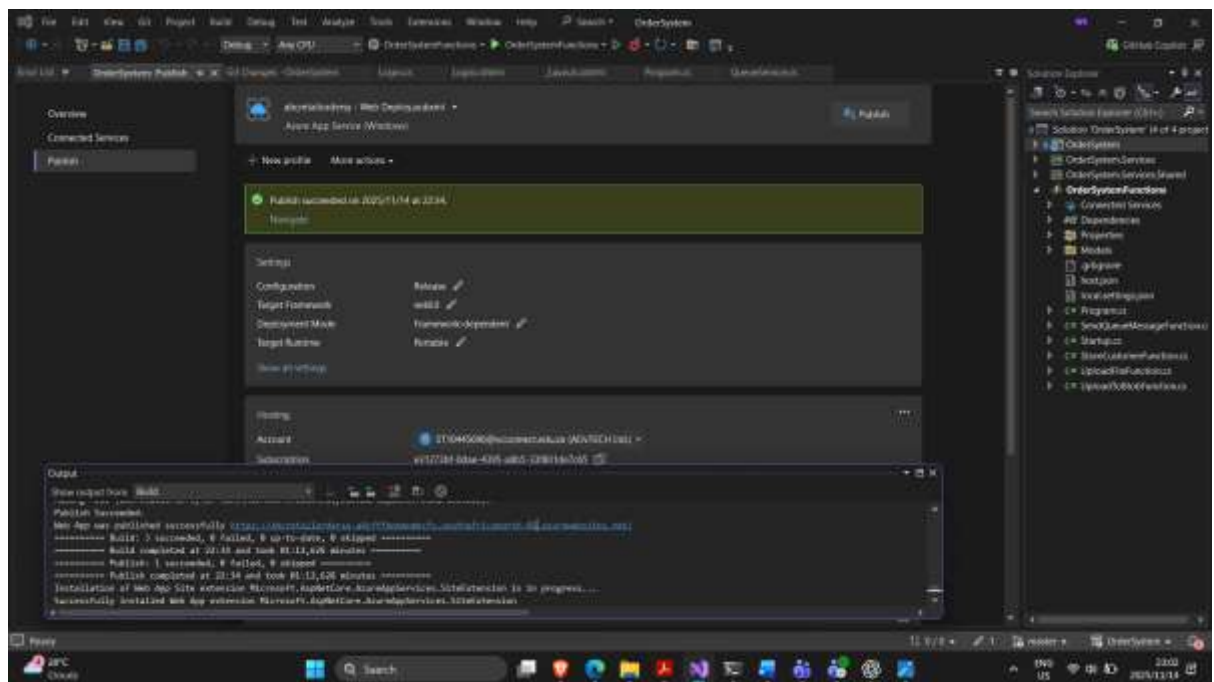
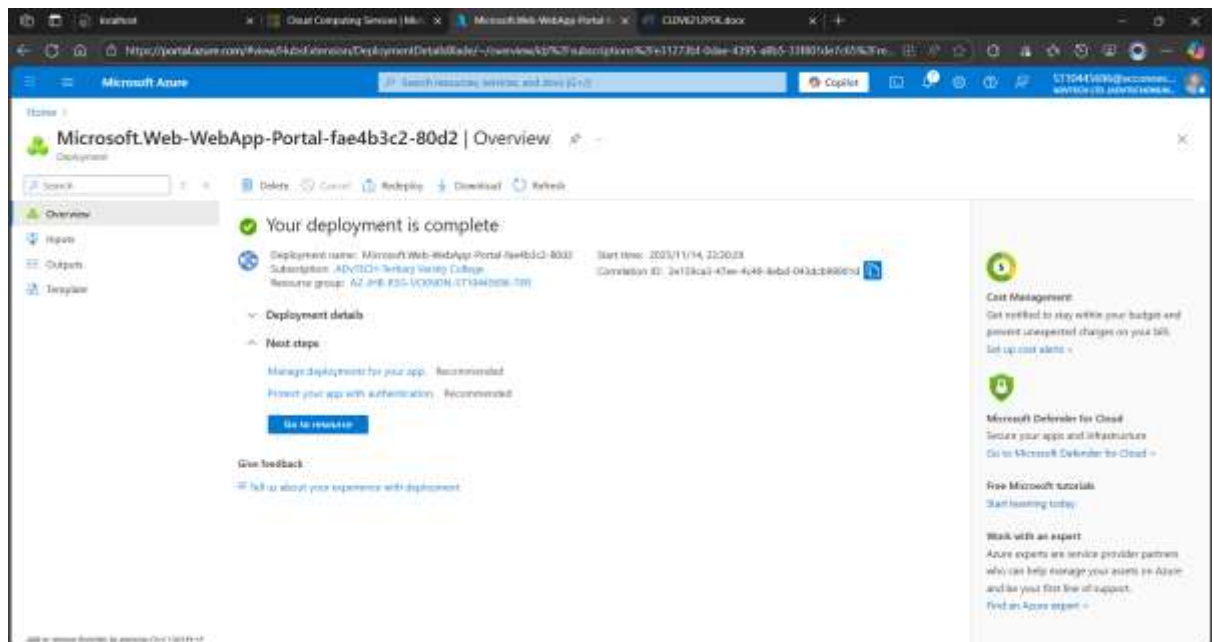
3. Original database



Why Creating a Replica Database in a Different Region Is Necessary

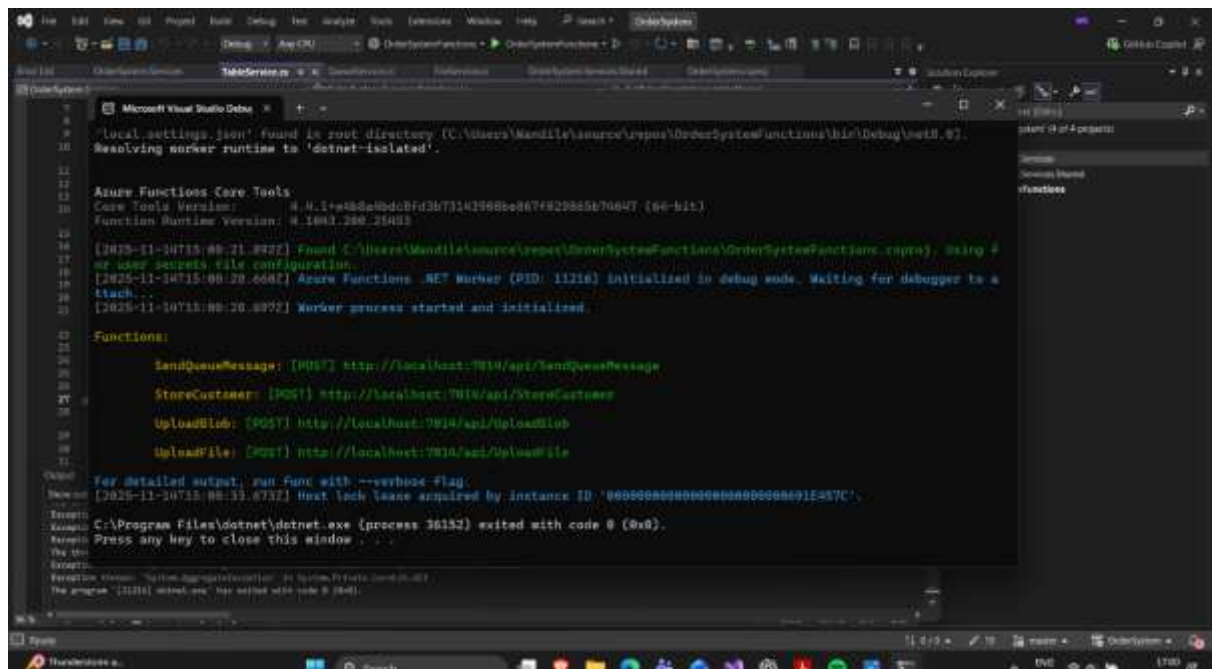
One key tactic for enhancing the dependability, efficiency, and resilience of contemporary systems is the creation of a geographically distinct replica of a database. System activities can continue even in the event of a disaster, outage, or network failure in one location thanks to a multi-region database configuration. Cloud regions fail more frequently than companies anticipate, and disruptions can result in large operational and financial losses, claims Balagurunathan(2022). The system may automatically transition to the healthy region by keeping a second copy of the database in a different location, guaranteeing business continuity.

Azure App Service Deployment



Live web app URL: <https://abcetailordersy-a4cfffbphmcmbcfu.southafricanorth-01.azurewebsites.net/>

4. Improvement on Lecturer Feedback



The screenshot displays the Microsoft Visual Studio Debug Console window. The main pane shows the output of the Azure Functions Core Tools. The output includes the following information:

- Local settings.json found in root directory (C:\Users\Hendile\source\repos\OrderSystemFunctions\bin\Debug\WebHost).
- Resolving worker runtime to 'dotnet-isolated'.
- Azure Functions Core Tools version: 3.0.1+e488e0d0b3d373142988be867e29865b70647 (64-bit).
- Function Runtime Version: 3.1043.286.25403.
- Log messages indicating the worker process started and initialized.
- A list of functions: `SendQueueMessage`, `StoreCustomer`, `UploadBlob`, and `UploadFile`.
- Log messages indicating the worker process exited with code 0 (0x0).

The right-hand pane shows the Solution Explorer, displaying the project structure of the OrderSystemFunctions project.

Functions Work

B) Document Technology choices for your solution

Component / Service	Technology Choice	Hosting Model
Azure App Service (Web App)	Compute service used to host the ASP.NET Core MVC web application	PaaS (Platform as a Service)
Azure SQL Database	Relational data storage used for structured data migrated from Microsoft SQL Server Management Studio	PaaS
Microsoft SQL Server Management Studio (SSMS)	Local database development, query management, schema design, and migration tool used before publishing to Azure SQL	On-Premises / Local Tool
Azure Storage Account – Table Storage	NoSQL key-value data storage used for Login table, user authentication data and application metadata	PaaS
Azure Storage Explorer	Local client tool for managing Azure Table Storage entities during development	Client Tool (Local)
Azure Functions (Optional if used)	Serverless compute for background tasks, table operations, or triggers	FaaS (Function as a Service)
Azure Portal	Cloud-based management interface used to configure App Services, SQL Databases, and Storage Accounts	Cloud Management Interface
Visual Studio 2022	IDE used for developing the ASP.NET Core application, integrating Azure SDKs, and publishing the web app	Local Development Tool
GitHub / Git Source Control	Version control system used to track project development and deploy to Azure	SaaS (Software as a Service)
BCrypt.Net Library	Application-level password hashing for secure authentication	Application Dependency (Locally executed within App Service)

ASP.NET Core MVC Framework	Web application framework used to build controllers, views, models, and business logic	Framework (Executed in App Service – PaaS)
Azure App Service Deployment Center	CI/CD deployment mechanism from local machine or GitHub	PaaS Deployment Tool
Azure Monitor / App Insights (If Enabled)	Application monitoring, request tracking, and performance analytics	PaaS

C. Motivation of Azure Services Used for Application Functionality

ABC Retail's old web app just couldn't keep up anymore. Every peak shopping season, things would slow down images took forever to load, messages got lost, and there was no way to see what was happening in real time. The company needed something faster, more reliable, and easier to scale. So, they mapped out six key requirements and matched each one with the right Azure service, making sure everything actually fit the demands of a modern online store.

First up, they needed a place to host a secure, scalable web app built with ASP.NET Core MVC supporting session-based authentication, Razor views, and dynamic routing. They went with Azure App Service. This platform just works: it runs .NET 8 out of the box, handles scaling and load balancing on its own, and lets you update the app without any downtime. When traffic spikes during the holidays, App Service automatically spins up more resources, so customers never see a slowdown. Plus, it connects easily with Azure Active Directory, lets you use your own domain, and handles SSL without fuss. Compared to babysitting old IIS servers, it's just less hassle and way more secure.

Next, for storing all that customer, order, and product information, they needed a solid relational database with strong integrity and fast queries. Azure SQL Database fit the bill. It's fully managed and SQL Server compatible, so everything just transfers over. Features like automatic backups, geo-replication, and point-in-time restore mean data's always safe. The Hyperscale tier can handle thousands of transactions at once, perfect for those busy shopping days. And with high availability and automated failover, order data stays consistent which is huge for both customers and the bottom line.

For login functionality, speed matters. So, instead of storing authentication info in a bulky relational table, they switched to Azure Table Storage. It's a NoSQL key-value store, built for fast lookups and massive scale. With the right structure (PartitionKey as "Login," RowKey as a GUID), login checks are lightning-fast. It's cheap, efficient, and there's no need for complex joins or queries. This setup just makes sense for simple authentication data.

Product images used to sit on slow network drives. Now, they live in Azure Blob Storage. This service is durable, accessible over HTTP, and offers different redundancy options. Pairing it with Azure CDN means images load quickly, no matter where the customer is. Old images get moved automatically to cheaper storage tiers, and built-in versioning protects against accidental deletes. It's a major upgrade from the clunky old file servers.

Background tasks like sending order confirmation emails or updating inventory no longer drag down the system. Azure Queue Storage, combined with Azure Functions, handles these smoothly. Queues guarantee every message gets processed, and Functions jump into action as soon as new messages arrive. This keeps the main app responsive and prevents background work from clogging things up, especially when traffic surges.

Finally, real-time monitoring was a must. With Application Insights (part of Azure Monitor), ABC Retail can track exactly what's happening: failed logins, slow pages, weird exceptions it's all there. The platform's smart enough to spot issues before customers notice, and live metrics make it easy to see where things need tweaking. Custom dashboards and alerts mean the team can react fast and keep everything running smoothly.

In the end, each Azure service was picked to solve a specific pain point, making the new ABC Retail app faster, more reliable, and ready for anything shoppers throw its way.

D. Alternative Azure Services: Viable Options for Key Requirements

1. Web Hosting Alternative: Azure Container Apps

Instead of using App Service, ABC Retail can run the MVC app as a Docker container with Azure Container Apps. This setup isn't just about running containers you get KEDA for scaling (say, based on queue length), Dapr for state management, and easy revision-based deployments. If ABC Retail moves toward microservices, like splitting out Product and Order APIs, Container Apps really makes sense. It does mean you'll need to keep your Dockerfile up to date, but you gain a lot more control over the runtime and you're not locked into Azure you can move to AWS or even run on your own Kubernetes cluster if you want. Session affinity works through cookies, though it isn't quite as smooth as the built-in session management in App Service (Microsoft, 2025).

2. Relational Data Alternative: Azure Cosmos DB (SQL API)

Azure Cosmos DB with the SQL API gives you full SELECT...JOIN support, plus multi-region writes, automatic indexing, and elastic throughput. For something like global inventory sync think Johannesburg and London warehouses Cosmos DB can copy data across regions in under 15ms, which beats Azure SQL's geo-replication. You also get a change feed for real-time analytics. It costs more, but you don't have to deal with database sharding, and it scales all the way up to petabyte-sized datasets (Foulkes, 2023)

3. Authentication Data Alternative: Azure Cosmos DB (Table API)

With Cosmos DB's Table API, you get all the same SDKs and query patterns as Table Storage, but with more features. Multi-region replication, TTL, and an analytical store are built in. So, if ABC Retail needs users to log in from anywhere maybe failover to the EU if South Africa goes down or wants to analyze login trends with Synapse Link, it can do all that without rewriting code. Plus, you get premium SLAs and automatic backups (Cope, 2024).

4. Message Queuing Alternative: Azure Service Bus

Azure Service Bus brings in FIFO queues, sessions, duplicate detection, and transactions, which is huge for payment workflows that have to be processed exactly once. You can tie messages together for order tracking, and dead-lettering comes with clear reason codes. Queue Storage is cheaper, sure, but Service Bus is the way to go for anything financial where you just can't afford to lose a message. It works with Azure Functions through premium triggers and supports AMQP, so you can connect from just about anywhere (Microsoft Docs, 2025).

Conclusion

The recommended Azure setup App Service, Azure SQL, Table plus Blob Storage, Queue plus Functions, and Application Insights gives ABC Retail a scalable, secure, and reliable e-commerce platform that really addresses their old system's problems. Each service was picked for managed operations, cost savings, and solid performance. But with alternatives like Container Apps, Cosmos DB, and Service Bus, ABC Retail can stay flexible and ready to grow as the business changes.

References

Balagurunathan, K. (2022) 4 Reasons Why Your Database Should Be Multi-Region. [Online] Available at: <<https://dev.to/datastax/4-reasons-why-your-database-should-be-multi-region-medium-synd-aib>> [Accessed 13 November 2025].

Cope, J. (2024). Azure Storage Deep Dive: When to Use Blob, Table, or Cosmos DB. [Online] Available at: <<https://learn.microsoft.com/en-us/azure/storage/>> [Accessed 13 November 2025].

Foulkes, M. (2023). Building Scalable Systems with Azure Queue Storage and Functions. [Online] Available at: <<https://learn.microsoft.com/en-us/azure/azure-functions/>> [Accessed 13 November 2025].

Microsoft. (2025). Azure App Service Documentation. [Online] Available at: <<https://learn.microsoft.com/en-us/azure/app-service/>> [Accessed 13 November 2025].

Microsoft Docs. (2025). Azure SQL Database Overview. [Online] Available at: <<https://learn.microsoft.com/en-us/azure/azure-sql/>> [Accessed 13 November 2025].

Microsoft Learn. (2025). Application Insights Overview. [Online] Available at: <<https://learn.microsoft.com/en-us/azure/azure-monitor/app/app-insights-overview>> [Accessed 13 November 2025].